Retro Graphics Converter — Project Audit Report

Date: March 7, 2026 **Project:** Retro Graphics Converter (macOS SwiftUI App) **Codebase:** ~24,150 lines of Swift across 42 source files **Author of Report:** Claude (Automated Audit)

Table of Contents

1. Executive Summary
2. Project Overview
3. Architecture Analysis
4. Critical Issues
5. Code Quality Concerns
6. Performance Issues
7. Security & Robustness
8. UI/UX Observations
9. Maintainability & Scalability
10. Recommendations Summary

1. Executive Summary

The Retro Graphics Converter is a well-crafted macOS application for decoding and viewing retro computer graphics formats. It supports an impressive range of platforms (Apple II, C64, Amiga, Atari ST, ZX Spectrum, BBC Micro, MSX, TRS-80, and more) with palette editing, export, and movie creation features.

Overall Assessment: GOOD with areas for improvement.

The codebase demonstrates strong domain knowledge and generally solid engineering. However, the project would benefit from architectural refactoring to improve maintainability, better error handling, and some performance optimizations. The most pressing concerns involve the monolithic ContentView (2,163 lines), duplicated logic across decoders, and missing test coverage.

2. Project Overview

File Structure

Category	Files	Total Lines
App Entry	1	207
Models	4	1,128
Views	12	4,871
Decoders	19	14,726
Utilities	3	220
Other (ImageCropView)	1	298
Total	**40**	**~24,150**

Supported Formats

The app supports 20+ retro graphics platforms and 10+ disk image formats, which is an impressive scope. Modern image formats (PNG, JPEG, GIF, TIFF, HEIC) are also supported for export.

3. Architecture Analysis

Strengths

- **Clear separation of Decoders**: Each platform has its own decoder file (C64Decoder, AtariSTDecoder, BBCMicroDecoder, etc.), making format-specific logic easy to locate.
- **Consistent rendering pipeline**: All decoders converge on `ImageHelpers.createCGImage()` for final image output.
- **Well-structured model layer**: `AppleIIImageType`, `PaletteInfo`, and `DiskCatalog` provide good abstractions.
- **MARK comments**: Consistent use of `// MARK: -` for code organization.

Weaknesses

- **No protocol-based decoder interface**: Decoders are standalone static classes without a shared protocol. There is no `RetroDecoder` protocol with a standard `decode(data:) -> CGImage?` signature. This makes it harder to add new formats or test decoders uniformly.
- **Monolithic ContentView**: At 2,163 lines, `ContentView.swift` handles state management, file I/O, image transformations, palette editing, video encoding, and UI layout all in one file.
- **No dedicated ViewModel layer**: Business logic is mixed directly into SwiftUI views using `@State` properties rather than extracted into ObservableObject view models.
- **Tight coupling in SHRDecoder**: The main entry-point decoder (`SHRDecoder.swift`) acts as a giant router with extensive if/else chains based on file extensions and magic bytes, making it fragile to extend.

4. Critical Issues

4.1 — ContentView is a "God Object" (HIGH)

`ContentView.swift` at 2,163 lines contains:

- All application state (~30+ @State properties)
- File import/export logic
- Image transformation logic (rotate, flip, crop, invert)
- Brightness/contrast adjustment logic
- Palette editing and re-rendering logic
- Movie creation with AVAssetWriter
- Screensaver export
- Drag-and-drop handling
- Thumbnail grid management

Risk: Any change to one feature risks breaking another. This is the single biggest maintainability problem in the codebase.

Recommendation: Extract into at least 4–5 dedicated ViewModels: `ImageImportViewModel`, `ExportViewModel`, `TransformationViewModel`, `PaletteEditingViewModel`, `MovieExportViewModel`.

4.2 — No Unit Tests (HIGH)

The Xcode scheme references two test bundles, but no test files were found in the repository. The decoders are pure functions operating on binary data — they are ideal candidates for unit testing.

Risk: Regressions when modifying any decoder or adding new format support. Binary format parsing is notoriously easy to break.

Recommendation: Add unit tests for every decoder using known-good sample files. At minimum, test boundary conditions (empty data, truncated files, corrupt headers).

4.3 — Force-Unwrapped URL in App Entry (MEDIUM)

In `SHRConverterApp.swift` line 201:

```
URL(string: "https://github.com/portwally/Retro-Graphics-converter")!
```

Risk: If the string were ever malformed, this would crash the app at launch.

Recommendation: Use a `guard let` or provide a fallback.

5. Code Quality Concerns

5.1 — Duplicated LZW Decompression

LZW decompression for DreamGrafix appears in both `PaletteExtractor.swift` and `DreamGrafixDecoder.swift` (and is referenced in `PaletteRenderer.swift`). This is the same algorithm copy-pasted across files.

Recommendation: Extract into a single `LZWDecompressor` utility class.

5.2 — Mixed Language in Comments

Comments alternate between English and German throughout the codebase. Examples:

- `ImageTypes.swift` line 20: `"wird dynamisch basierend auf Dateityp gesetzt"`
- `DiskCatalog.swift` line 128: `"Initializer mit optionaler ID – wenn keine ID übergeben wird..."`
- `DiskCatalog.swift` line 160: `"Versuche die Original-Dateierweiterung zu ermitteln"`

Recommendation: Standardize all comments to English for consistency and to make the project more accessible to contributors.

5.3 — Inconsistent Error Handling

Most decoders return `nil` on failure without logging why. For example, `SHRDecoder.decode()` silently returns `nil` for unrecognized formats. The user gets no feedback about why an image failed to load.

Recommendation: Introduce a `Result<CGImage, DecoderError>` return type or at least add `os_log` / `Logger` calls for diagnostic information.

5.4 — Magic Numbers Throughout Decoders

Binary format parsing relies heavily on raw numeric literals:

- `32768` (SHR screen size)
- `10003` (C64 Koala file size)
- `9002`, `9009`, `10018` (C64 Art Studio sizes)
- `0x0296` (ATR magic number)

Recommendation: Define named constants for all magic numbers and file sizes to improve readability.

5.5 — Naming Inconsistency

The enum is called `AppleIIImageType` but it represents images from C64, Amiga, Atari, BBC Micro, MSX, and many non-Apple platforms. This is misleading.

Recommendation: Rename to `RetroImageType` or `RetroGraphicsFormat`.

6. Performance Issues

6.1 — Main Thread Image Processing

In `ContentView.swift`, palette re-rendering and image transformations appear to dispatch to background queues inconsistently. Some operations use `DispatchQueue.global(qos: .userInitiated)`, but others (like `applyAdjustments()`) operate on pixel buffers that may block the UI.

Recommendation: Audit all image processing paths and ensure they are consistently dispatched to background queues with proper `@MainActor` updates.

6.2 — Thumbnail Loading

All thumbnails are generated at full decode resolution and then scaled down in the view. For disk images containing dozens of files, this means decoding every image at full resolution just to show a thumbnail grid.

Recommendation: Implement a thumbnail cache with lazy loading. Decode at reduced resolution for thumbnails and only full-resolution for the selected preview.

6.3 — PaletteExtractor and PaletteRenderer Are Very Large

`PaletteExtractor.swift` (1,776 lines) and `PaletteRenderer.swift` (1,494 lines) are monolithic files with long switch statements. Each platform's palette extraction/rendering is in the same file.

Recommendation: Split into per-platform files or use a strategy pattern with platform-specific palette handlers.

6.4 — Movie Export Memory Usage

The movie export creates pixel buffers for every frame in memory during transition rendering. For 4K output with many images, this could consume significant RAM.

Recommendation: Process frames sequentially and release pixel buffers immediately after encoding each frame.

7. Security & Robustness

7.1 — Binary Parsing Without Sufficient Bounds Checking

While many decoders do check data sizes, some paths could be more defensive:

- Disk readers trust directory entries for file sizes without capping them
- Some decoders access array indices based on file content without full validation

Recommendation: Add systematic bounds checking on all data access. Consider using a `SafeDataReader` wrapper that returns `nil` instead of crashing on out-of-bounds access.

7.2 — Disk Image Parsing Attack Surface

The disk image readers (ProDOS, DOS 3.3, FAT12, DFS, TR-DOS, etc.) parse untrusted binary data. Maliciously crafted disk images could potentially trigger:

- Integer overflows in size calculations
- Excessive memory allocation from inflated size fields
- Infinite loops in FAT chain traversal

Recommendation: Add maximum allocation limits and cycle detection in FAT chain following. Cap file sizes to reasonable maximums (e.g., disk image total size).

7.3 — No Sandboxing Indicators

The project does not appear to include an entitlements file for App Sandbox. For a file-handling app distributed outside the App Store, this is acceptable, but for Mac App Store distribution, sandboxing would be required.

Recommendation: If targeting the Mac App Store, add proper sandbox entitlements with file access scopes.

8. UI/UX Observations

8.1 — No Loading Indicators for Heavy Operations

Batch export, movie creation, and disk image parsing can take significant time but the UI feedback appears limited. The status bar shows counts but not progress percentage.

Recommendation: Add a progress bar or spinner for long-running operations, especially movie export and batch processing.

8.2 — Export Sheet Could Be More Intuitive

The `ExportSheet` has a mutual exclusivity pattern where selecting "Original" clears modern format selections, but this behavior may not be obvious to users.

Recommendation: Use a radio-button style or segmented control to make the exclusivity clearer.

8.3 — Help View Is Comprehensive but Static

The HelpView (767 lines) is entirely hardcoded. Any format addition requires updating the help text manually.

Recommendation: Consider generating supported format lists dynamically from the decoder registry.

8.4 — ImageCropView is in the Root Directory

`ImageCropView.swift` sits at the project root level rather than in the `Views/` folder.

Recommendation: Move to `SHRConverter/Views/ImageCropView.swift` for consistency.

9. Maintainability & Scalability

9.1 — Adding a New Format Requires Changes in 4+ Files

To add support for a new retro platform, a developer must currently modify:

1. `SHRDecoder.swift` (format routing)
2. New decoder file
3. `PaletteExtractor.swift` (palette support)
4. `PaletteRenderer.swift` (live palette preview)
5. `ImageTypes.swift` (enum case)
6. `HelpView.swift` (documentation)
7. `UTTypeExtensions.swift` (file type registration)

Recommendation: Implement a decoder registry pattern where each decoder self-registers its supported extensions, magic bytes, and capabilities. This would reduce the number of files that need modification for new formats.

9.2 — No Dependency Management

The project uses no external dependencies (SPM, CocoaPods, or Carthage). While this reduces complexity, some well-established libraries could improve code quality:

- **swift-log** for structured logging
- **swift-collections** for ordered sets/dictionaries

9.3 — Version and Build Info

There's no visible version number or build information in the app. The CHANGELOG.md exists but there's no programmatic access to version info.

Recommendation: Display version info in the Help or About section.

10. Recommendations Summary

Priority 1 — Critical (Address Soon)

#	Issue	Impact
1	Break up ContentView into ViewModels	Maintainability
2	Add unit tests for all decoders	Reliability
3	Standardize error handling with logging	Debuggability

Priority 2 — Important (Next Sprint)

#	Issue	Impact
4	Extract duplicated LZW code	Code quality
5	Rename AppleIIImageType to RetroImageType	Clarity
6	Define named constants for magic numbers	Readability
7	Add bounds-checking wrapper for binary parsing	Security
8	Standardize comments to English	Accessibility

Priority 3 — Recommended (Backlog)

#	Issue	Impact
9	Implement decoder registry/protocol	Extensibility
10	Add thumbnail caching and lazy loading	Performance
11	Split PaletteExtractor/Renderer by platform	Maintainability
12	Add progress indicators for long operations	UX
13	Move ImageCropView.swift into Views folder	Organization
14	Add version info display	Polish
15	Review movie export memory usage	Performance

This audit was performed by analyzing all 40 Swift source files (approximately 24,150 lines of code) in the Retro Graphics Converter project. The findings are based on static code review and architectural analysis.